Jiaxin Huang, Shixiang Shane Gu, Le Hou, Yuexin Wu, Xuezhi Wang, Hongkun Yu, and Jiawei Han. 2022. Large language models can self-improve. *arXiv preprint arXiv:2210.11610*.

Martin Huschens, Martin Briesch, Dominik Sobania, and Franz Rothlauf. 2023. Do you trust chatgpt?–perceived credibility of human and ai-generated content. *arXiv preprint arXiv:2309.02524*.

Dan Jurafsky and James H Martin. 2009. *Speech and language processing: An introduction to natural language processing, computational linguistics, and speech recognition*. Prentice Hall, Pearson Education, Upper Saddle River, NJ, USA.

Sarah Kreps, R Miles McCain, and Miles Brundage. 2022. All the news that's fit to fabricate: Ai-generated text as a tool of media misinformation. *Journal of experimental political science*, 9(1):104–117.

Vladimir I Levenshtein et al. 1966. Binary codes capable of correcting deletions, insertions, and reversals. In *Soviet physics doklady*, volume 10, pages 707–710. Soviet Union.

Weixin Liang, Yaohui Zhang, Zhengxuan Wu, Haley Lepp, Wenlong Ji, Xuandong Zhao, Hancheng Cao, Sheng Liu, Siyu He, Zhi Huang, et al. 2024. Mapping the increasing use of llms in scientific papers. *arXiv preprint arXiv:2404.01268*.

Chin-Yew Lin. 2004. Rouge: A package for automatic evaluation of summaries. In *Text summarization branches out*, pages 74–81.

Gonzalo Martínez, Lauren Watson, Pedro Reviriego, José Alberto Hernández, Marc Juarez, and Rik Sarkar. 2023. Towards understanding the interplay of generative artificial intelligence and the internet. *arXiv preprint arXiv:2306.06130*.

Kishore Papineni, Salim Roukos, Todd Ward, and Wei-Jing Zhu. 2002. Bleu: a method for automatic evaluation of machine translation. In *Proceedings of the 40th annual meeting of the Association for Computational Linguistics*, pages 311–318.

Justin K Pugh, Lisa B Soros, Paul A Szerlip, and Kenneth O Stanley. 2015. Confronting the challenge of quality diversity. In *Proceedings of the 2015 Annual Conference on Genetic and Evolutionary Computation*, pages 967–974.

Vinu Sankar Sadasivan, Aounon Kumar, Sriram Balasubramanian, Wenxiao Wang, and Soheil Feizi. 2023. Can ai-generated text be reliably detected? *arXiv preprint arXiv:2303.11156*.

Ilia Shumailov, Zakhar Shumaylov, Yiren Zhao, Yarin Gal, Nicolas Papernot, and Ross Anderson. 2023. The curse of recursion: Training on generated data makes models forget. *arXiv preprint arXiv:2305.17493*.

Dominik Sobania, Martin Briesch, Carol Hanna, and Justyna Petke. 2023. An analysis of the automatic bug fixing performance of chatgpt. In *2023 IEEE/ACM International Workshop on Automated Program Repair (APR)*, pages 23–30, Los Alamitos, CA, USA. IEEE Computer Society.

Dominik Sobania, Martin Briesch, and Franz Rothlauf. 2022. Choose your programming copilot: A comparison of the program synthesis performance of github copilot and genetic programming. In *Proceedings of the genetic and evolutionary computation conference*, pages 1019–1027.

Emma Strubell, Ananya Ganesh, and Andrew McCallum. 2019. Energy and policy considerations for deep learning in nlp. *arXiv preprint arXiv:1906.02243*.

Veniamin Veselovsky, Manoel Horta Ribeiro, and Robert West. 2023. Artificial artificial artificial intelligence: Crowd workers widely use large language models for text production tasks. *arXiv preprint arXiv:2306.07899*.

Pablo Villalobos, Jaime Sevilla, Lennart Heim, Tamay Besiroglu, Marius Hobbhahn, and Anson Ho. 2022. Will we run out of data? an analysis of the limits of scaling datasets in machine learning. *arXiv preprint arXiv:2211.04325*.

Yizhong Wang, Yeganeh Kordi, Swaroop Mishra, Alisa Liu, Noah A Smith, Daniel Khashabi, and Hannaneh Hajishirzi. 2022. Self-instruct: Aligning language model with self generated instructions. *arXiv preprint arXiv:2212.10560*.

David Wittenberg, Franz Rothlauf, and Christian Gagné. 2023. Denoising autoencoder genetic programming: strategies to control exploration and exploitation in search. *Genetic Programming and Evolvable Machines*, 24(2):17.

Xianjun Yang, Yan Li, Xinlu Zhang, Haifeng Chen, and Wei Cheng. 2023. Exploring the limits of chatgpt for query or aspect-based text summarization. *arXiv preprint arXiv:2302.08081*.

Yue Yu, Yuchen Zhuang, Jieyu Zhang, Yu Meng, Alexander Ratner, Ranjay Krishna, Jiaming Shen, and Chao Zhang. 2023. Large language model as attributed training data generator: A tale of diversity and bias. *arXiv preprint arXiv:2306.15895*.

Tianyi Zhang, Varsha Kishore, Felix Wu, Kilian Q Weinberger, and Yoav Artzi. 2019. Bertscore: Evaluating text generation with bert. *arXiv preprint arXiv:1904.09675*.

A Generation of the Logic Expression Dataset

Algorithm 2 describes the generation of our logic expression dataset in detail.

Algorithm 2 Generate Logic Expression Dataset

Input: number of unique expressions m, minimum tree depth d_{min}, maximum tree depth d_{max}
Initialize: $\mathcal{D}_0 = \{\}$

function GenLogicExpr(d) :
if $d = 0$ **then**
  return RandomBool()
else
  $op \leftarrow$ RandomLogicOperator()
  if $op =$ 'not' **then**
    return $op +$ GenLogicExpr($d-1$)
  else
    $leftExpr \leftarrow$ GenLogicExpr($d-1$)
    $rightExpr \leftarrow$ GenLogicExpr($d-1$)
    return $leftExpr + op + rightExpr$
  end if
end if
end function

while $|\mathcal{D}_0| < m$ **do**
  $i \leftarrow$ RandomIntegerInRange(d_{min}, d_{max})
  $expr \leftarrow$ GenLogicExpr($d = i$)
  if EvalBoolExpr($expr$) = True **then**
    $\mathcal{D}_0 \leftarrow \mathcal{D}_0 \cup \{expr\}$
  end if
end while

B Unique Expressions Generated

To better understand the decline of diversity, we inspect the number of unique expressions sampled at each generation. We focus on the full synthetic data cycle, as this data cycle fully collapses to a single point within 50 generations.

Figure 7 displays the number of unique expressions (right y-axis) generated at each generation of a self-consuming training loop in comparison to the diversity measured as the average pairwise normalized Levenshtein distance (left y-axis). We observe that in the beginning the number of unique expressions stays stable while the diversity is already in a steep decline. Only by generation $t = 10$ the number of unique individuals starts to decline rapidly and by generation $t = 15$ only around 400 unique expressions are sampled while the decline

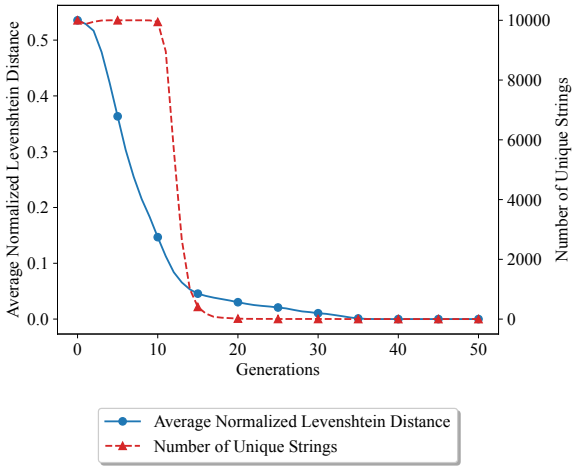

Figure 7: Average pairwise normalized Levenshtein distance (left y-axis) and number of unique expressions in a sample (right y-axis) over generations of a self-consuming training loop for the full synthetic data cycle. Line markers added every 5 generations for better display.

in diversity slows down. In generation $t = 21$ the number of unique individuals is below 10 and in generation $t = 39$ only a single individual is sampled at each generation. Consequently, the diversity is collapsed to zero.

This shows that it is not enough to only track unique outputs of a model, but that diversity needs to be tracked as well. By the time a decrease in unique results is noticeable, the diversity already decreased significantly.

C Impact of Initialization

So far we only looked at the self-consuming training loop starting from an initial dataset consisting only of `True` expressions. To check whether our findings also hold for different initializations of data across multiple runs, we conduct further experiments. Specifically, we analyze the full synthetic data cycle with the original dataset $\mathcal{D}_0$ only consisting of (1) `True` expressions, (2) `False` expressions, and (3) an equal mixture of both. The results from Sect. 4 indicate that the effect of the self-consuming training loop for the full synthetic data cycle are already clearly visible in the first 10 generations. Therefore, we conduct experiments with only 10 generations. With this saved computing time, we conduct 5 independent runs for each initialization.

Figure 8 displays the composition of samples $\mathcal{S}_t$ generated during a self-consuming training loop.

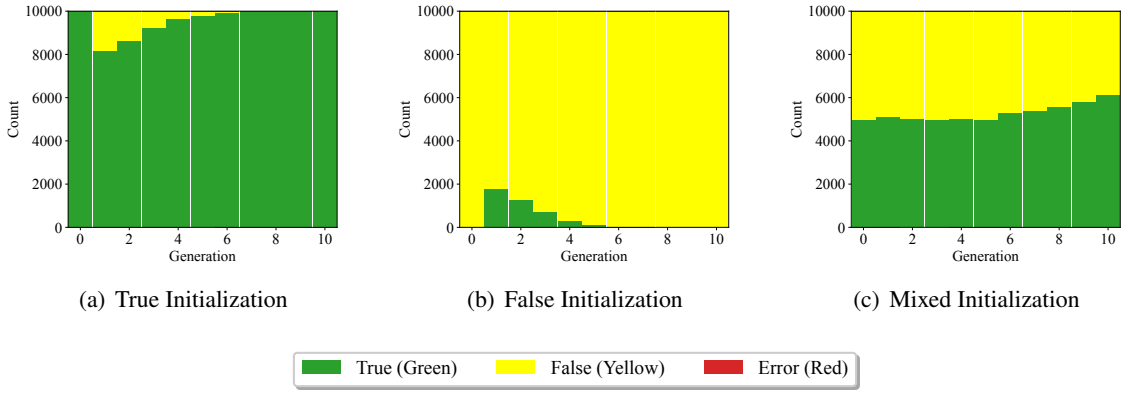

Figure 8: The composition of each sample $\mathcal{S}_t$ from model $\mathcal{M}_t$ at generation t (the first bar displays the composition of $\mathcal{D}_0$) with regards to the number of syntactically and semantically correct expressions averaged over 5 runs. Green indicates syntactically correct expressions that evaluate to `True`, yellow indicates syntactically correct expressions that evaluate to `False`, and red indicates syntactically incorrect expressions that result in an error when being parsed. Each subplot displays the results for the full synthetic data cycle with different initializations (`True`, `False`, Mixed).

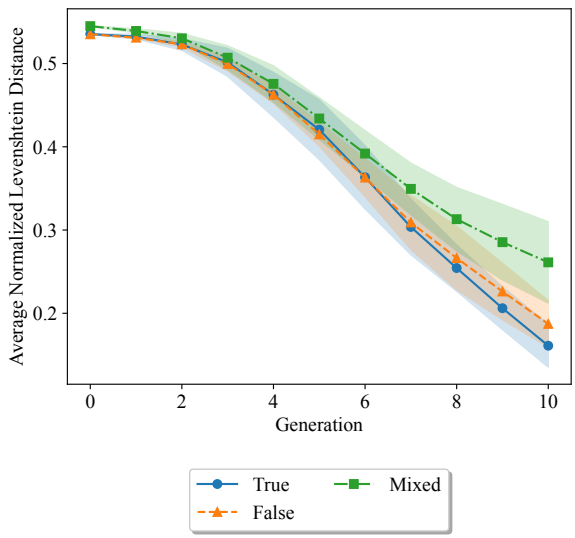

Figure 9: Average pairwise normalized Levenshtein distance over generations of a self-consuming training loop for the full synthetic data cycle with different initializations. Displayed is the mean and standard deviation at each generation for each initialization over 5 runs.

Each subplot presents the composition for a different initialization for the full synthetic data cycle averaged over 5 independent runs: a) only `True` expressions, b) only `False` expressions, and c) half `True` expressions and half `False` expressions. Every bar in a subplot displays the composition of $\mathcal{S}_t$ generated from model $\mathcal{M}_t$ at generation t, except for the first bar in each subplot which displays $\mathcal{D}_0$. The green portion of a bar indicates the number of syntactically correct expressions that evaluate to `True` in that sample. The yellow part of a bar represents the number of syntactically correct expressions that evaluate to `False`, and the red part of a bar displays the number of syntactically incorrect expressions that result in an error when being parsed.

Similar to our results in Sect. 4.1, we observe an initial drop of `True` expressions by 20% in favor of `False` expressions for the initialization with only `True` expressions in the first generation. Afterwards, the number of `True` expressions in a sample increases again over the course of generations. For the initialization with only `False` expressions, we observe a similar effect, only in the opposite direction. The number of `False` expression first drops by around 20% and then increases again until the whole sample consists of only `False` expressions. When initialized with an equal amount of `True` and `False` expressions, we can observe that the proportion of both expression types first stays stable and then by generation $t = 6$ starts to slightly shift towards `True` expressions. Upon further inspection of the 5 runs, we find that one run quickly shifts towards only `True` expressions, one run shifts quickly to only `False` expression, one run shifts slowly to `True` expressions, and two runs maintain equal proportions between `True` and `False` expressions across the 10 generations.

Figure 9 displays the diversity measured in the average pairwise normalized Levenshtein distance for the three different initializations. Displayed is the mean and standard deviation of five runs over the number of generations. We observe that the